

UNIT V: ACCESS CONTROL (5 HRS.)

5.1. Access Control Principles

5.2. Subjects, Objects and Access Rights

5.3. Access Control Matrix and Capability Lists

5.4. Discretionary Access Control

5.5. Role Based Access Control

5.6. Attribute Based Access Control

5.7. Identity, Credential and Access Management

5.8. Trust Frameworks

ACCESS CONTROL

- **Access control** is a security mechanism that regulates and restricts access to system resources by ensuring that only authorized users, processes, or entities can access specific resources, and only in permitted ways, according to a defined security policy.
- Access control determines:
 - **Who (user/process)** can access
 - **What resource** they can access
 - **What type of access** (read, write, execute)

OBJECTIVES OF ACCESS CONTROL

- Access control is a **core part of computer security**, with three main goals:
- Prevent **unauthorized users** from accessing resources
- Prevent **authorized users** from misusing access
- Ensure **authorized users get proper access** when needed

PRINCIPLES OF ACCESS CONTROL

- **Policy Enforcement**
Access control is based on a defined **security policy** that specifies who can access which resources and in what manner.
- **Authentication First**
Every user or entity must be **properly authenticated** before access is considered.
- **Authorization of Access**
After authentication, access is granted only if the user has the **required permissions** for the requested resource.
- **Least Privilege**
Users should be given the **minimum level of access** necessary to perform their tasks.
- **Separation of Duties**
Responsibilities should be divided among different individuals to **reduce risk of misuse or fraud**.
- **Controlled Access to Resources**
Access must be restricted to **authorized users, processes, and devices only**.
- **Limitation of Actions**
Users can perform only those **actions or operations** that they are permitted to execute.
- **Accountability and Auditing**
All access and activities should be **monitored and recorded** to detect and prevent security breaches.
- **Session Control**
User sessions should be **limited and automatically terminated** after inactivity or defined conditions.
- **Protection of Remote and External Access**
Remote, wireless, and external system access must be **secured using authentication, encryption, and control mechanisms**.

HOW ACCESS CONTROL WORKS

Access control operates along with three key functions:

1. Authentication

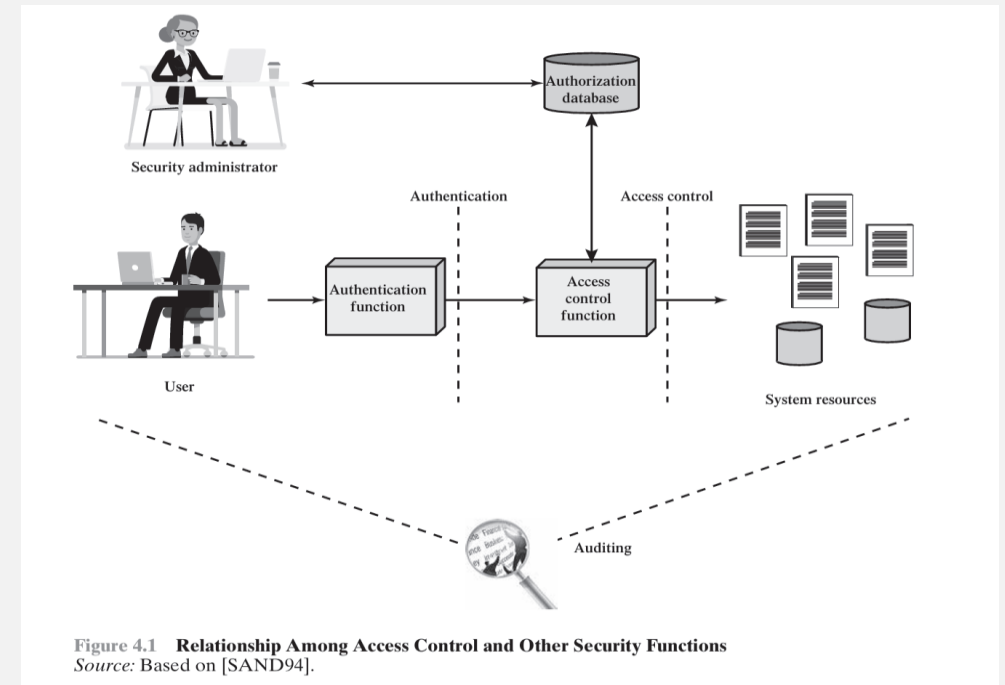
- Verifies identity (e.g., username/password)
- Confirms the user is genuine

2. Authorization

- Grants' permissions after authentication
- Decides what actions the user is allowed to perform

3. Auditing

- Records and monitors system activity
- Helps detect security violations and ensure compliance



SECURITY REQUIREMENTS

Basic Requirements

- Allow access only to **authorized users/processes/devices**
- Restrict users to **permitted actions only**

Derived Requirements (Key Ideas)

- Apply **least privilege principle**
- Separate duties to reduce risk
- Limit failed login attempts
- Use **session locks and automatic logout**
- Secure **remote and wireless access**
- Protect data using **encryption**
- Control **mobile devices and external systems**
- Monitor and audit all access activities

SUBJECTS, OBJECTS, AND ACCESS RIGHTS

Access control is based on three fundamental elements: **subject, object, and access rights**.

Subject

- A **subject** is any entity that can access a resource (object).
- It is usually a **process** acting on behalf of a user or application.
- The subject carries the **permissions and attributes** of the user.
- It is **accountable** for its actions, which can be tracked using an **audit trail**.

Types of Subjects

- **Owner**: The creator of the resource (e.g., file creator or system administrator). Has full control.
- **Group**: A set of users who share common access rights. A user can belong to multiple groups.
- **World (Others)**: All other users who have minimal or limited access.

SUBJECTS, OBJECTS, AND ACCESS RIGHTS...

Object

- An **object** is any resource that needs protection and access control.
- It is something that stores or receives information.

Examples of Objects

- Files, directories, databases
- Programs, messages, mailboxes
- System resources like memory, processors, network nodes

The type and number of objects depend on the system environment and the balance between **security and complexity**.

SUBJECTS, OBJECTS, AND ACCESS RIGHTS...

Access Rights

- **Access rights** define how a subject can interact with an object.

Common Access Rights

- **Read:** View or copy data
- **Write:** Modify or delete data (includes read access)
- **Execute:** Run programs
- **Delete:** Remove files or resources
- **Create:** Make new files or records
- **Search:** List or browse directory contents

ACCESS CONTROL MATRIX

- The **Access Control Matrix** is the fundamental theoretical model for Discretionary Access Control (DAC). It was originally formulated by **Butler Lampson** [LAMP69, LAMP71] and later refined by Graham & Denning and Harrison et al.

Structure of the Access Control Matrix

- It is a **two-dimensional table** (matrix).
 1. **Rows** represent **Subjects**
 - Subjects can be: individual users, user groups, processes, terminals, hosts, applications, etc.
 2. **Columns** represent **Objects** (what is being accessed).
 - Objects can be: files, records, data fields, directories, devices, memory regions, processes, or even other subjects.
- **Each cell** [Subject, Object] contains the **access rights** that the subject has for that object.
 - Examples of access rights: read, write, execute, own, control, delete, wakeup, seek, etc.

EXAMPLE OF ACCESS CONTROL MATRIX

Subjects \ Objects	File 1	File 2	File 3	File 4
User A	Own, Read, Write	-	Own, Read, Write	-
User B	Read	-	-	Write
User C	Read, Write	Own, Read, Write	-	Read

- User A **owns** File 1 and File 3 → can fully control them.
- User B has only **read** permission on File 1.
- User C has **read + write** on File 1 and **owns** File 2.
- This matrix completely defines the **protection state** of the system at any point in time.

CAPABILITY LISTS

Capability Lists are the **row-wise decomposition** of the access matrix.

Definition : “Decomposition by rows yields **capability tickets**. A capability ticket specifies authorized objects and operations for a particular user. Each user has a number of tickets...”

What a Capability List Contains

- For each **subject** (user or process), there is a list of:
 - i. Objects they can access
 - ii. The specific access rights they have for each object

CAPABILITY LIST EXAMPLE

User A's Capability List:

- File 1: Own, Read, Write
- File 3: Own, Read, Write

User B's Capability List:

- File 1: Read
- File 4: Write

User C's Capability List:

- File 1: Read, Write
- File 2: Own, Read, Write
- File 4: Read

Subjects \ Objects	File 1	File 2	File 3	File 4
User A	Own, Read, Write	-	Own, Read, Write	-
User B	Read	-	-	Write
User C	Read, Write	Own, Read, Write	-	Read

ACCESS CONTROL LIST (ACL)

- An **Access Control List (ACL)** is a method of access control where **each object (file/resource)** keeps a list of **who can access it and what they can do**.
Instead of focusing on users, ACL focuses on **resources (objects/Files)**.
- For each file, there is a list of:
 - Users (or groups)
 - Their permissions (read, write, execute)

Example

- **File1 ACL:**
 - User A → Read, Write
 - User B → Read
 - User C → No access
- **File2 ACL:**
 - User A → Read
 - User C → Write

How ACL works

When a user tries to access a file:

- i. System checks the **ACL of that file**
- ii. Finds the user in the list
- iii. If permission exists →  Allow
- iv. If not →  Deny

DISCRETIONARY ACCESS CONTROL (DAC)

- Discretionary Access Control (DAC) is an access control mechanism in which the **owner of a resource can decide and grant access permissions to other users at their own discretion.**
- Scheme in which an entity may enable another entity to access some resources
- Often provided using an access matrix.
 - One dimension consist of identified subjects that may attempt data access to resources
 - The other dimension lists the objects that may be accessed
- Each entry in the matrix indicates the access rights of particular subject for particular objects

HOW DAC WORKS (STEP-BY-STEP)

- Ram creates a file → Ram becomes the **owner**.
- Ram sets permissions:
 - Sita can read.
 - Hari cannot access.
- Later, Ram can **change** these permissions anytime.
- Ram can even **give Sita permission to share** the file with others.
- This ability to **propagate permissions** is a key feature of DAC.

MAIN COMPONENTS OF DAC

1. Subjects: Users or processes that request access.

- Example: User A, User B, a running program.

2. Objects: Resources to be protected.

- Files, folders, printers, databases.

3. Access Rights (Permissions): Typical rights:

- Read (R) , Write (W) , Execute (X) , Delete, Modify, etc.

4. Ownership: Every object has an **owner** who controls permissions.

5. Access Control List (ACL): Each object maintains a list specifying:

- **Who** can access it and **how**.

User	Read	Write	Execute
Ram	✓	✓	✗
Sita	✓	✗	✗
Hari	✗	✗	✗

ROLE BASED ACCESS CONTROL (RBAC)

- Traditional DAC system defines the access rights of individual users and groups. RBAC is based on:
 - Roles that user assume in a system (instead of their identity)
 - Role is a job function within an organization. A role will have specific access rights to one or more resources
 - Assign Access rights to Roles (instead of individual users)
 - Users assigned to different roles according to their responsibilities'
 - User-to-Roles are Many-to-Many
- The set of users change frequently, and the assignment of user to One or many roles is also dynamic
- The set of rules is relatively static, with only occasional addition or deletion.
- The set of Resources and the specific access right associated with a particular role are also likely to change infrequently which is relatively static.

ROLE BASED ACCESS CONTROL (RBAC)

- **Role-Based Access Control (RBAC)** is an access control model where **permissions are assigned to roles**, and **users are assigned to those roles**. Users don't get permissions directly—they **inherit permissions through their role**.
- “You access things because of your job role, not because you personally own them.”
- Instead of saying: “Ram can read File A”, RBAC says: “The **Accountant** role can read financial files” and “Ram is an **Accountant**” → so Ram can read them.

MAIN COMPONENTS OF RBAC

1. Users/Subjects

- Users are the People or processes that need access.
Example: Ram, Sita, Hari.

2. Roles

- Roles are the job functions in the organization.
Examples: Admin, Accountant, Manager, Clerk

3. Permissions

- Permissions are operations that are allowed on objects.
- Read, Write, Delete, Approve, Execute

4. Objects

- Objects are the Resources to be protected.
- Files, databases, applications, printers

HOW RBAC WORKS (STEP-BY-STEP)

- System defines roles: **Admin, Accountant, Clerk.**
- Each role is given permissions.
- Users are assigned roles.
- When a user logs in, the system checks **their role**, not their name.

User	Role	Permissions	Object
Ram	Accountant	Read	Accounts.xlsx
Sita	Clerk	Write	Records.docx
Hari	Admin	All (Read, Write, Delete, Execute)	Server

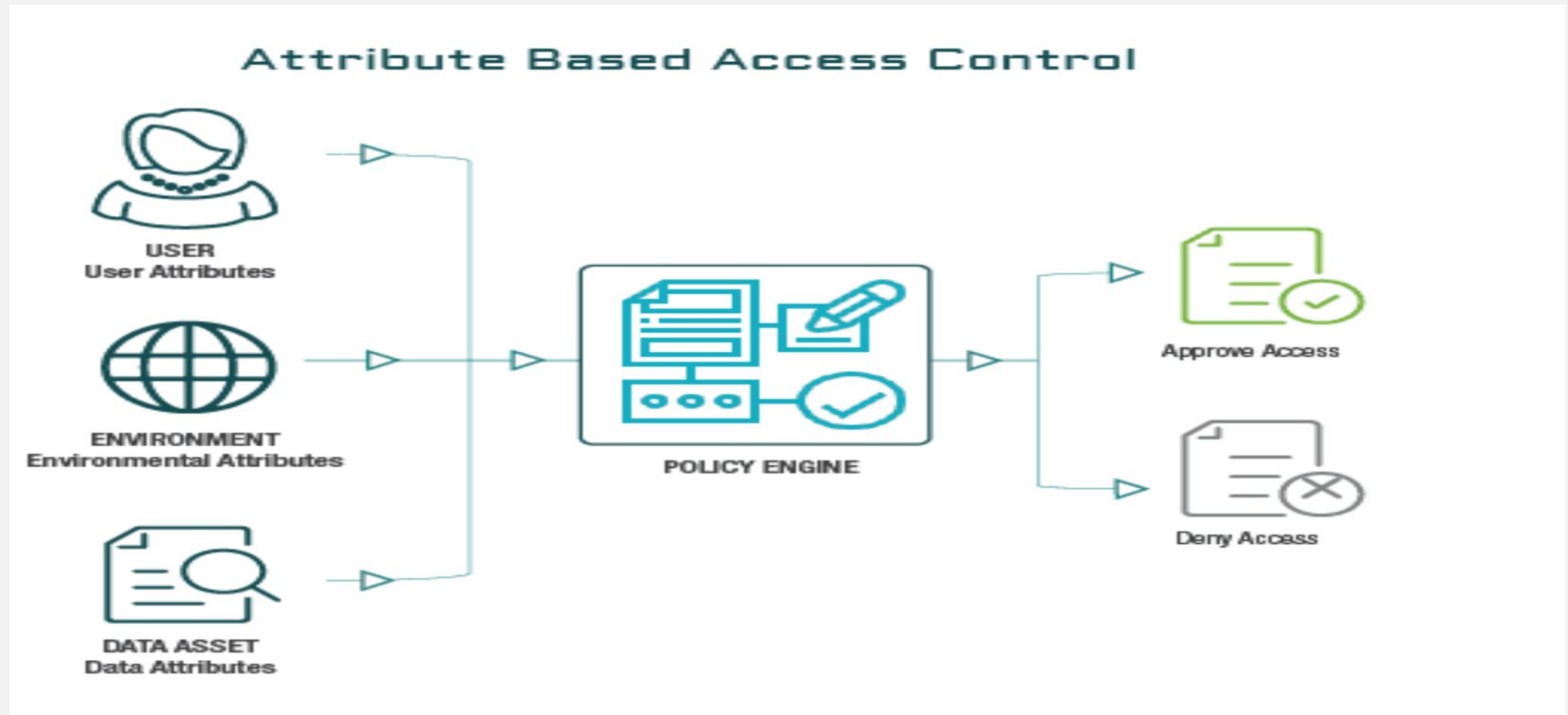
BEST PRACTICES OF RBAC

- RBAC allows to
 - Segregate duties within teams
 - Grant only the amount of access to users that they need to perform the jobs
- Instead of giving everybody or group unrestricted permissions on a resources, you can allow only certain actions at a particular scope.
- Planning the access control strategy.
 - Each role should contain minimum set of access rights needed for that role.
 - Its a best practice to grant user the least privilege to get their work done

ATTRIBUTE-BASED ACCESS CONTROL (ABAC)

- Attribute-Based Access Control (ABAC) is a modern authorization model that grants or denies access based on attributes associated with users, resources, actions, and environmental conditions.
- It provides dynamic and context-aware security decisions instead of relying only on fixed roles.
- ABAC enables fine-grained control, making it highly suitable for complex and large-scale systems.
- ABAC evaluates access requests by analyzing multiple attributes together rather than using a single role or identity.
- It uses logical policies that combine conditions such as “user role,” “resource type,” and “access time.”
- This allows organizations to enforce highly specific and flexible security rules.

ATTRIBUTE-BASED ACCESS CONTROL (ABAC)



TYPES OF ATTRIBUTES IN ABAC

- **User attributes** define who is requesting access and include details such as **role**, **department**, and **security clearance level**.
- **Resource attributes** describe what is being accessed, such as file type, ownership, and classification level.
- **Action attributes** define what operation is being performed, such as read, write, delete, or update.
- **Environment attributes** describe contextual conditions such as time, location, device type, and network security status.

USE CASES AND APPLICATIONS OF ABAC

- In healthcare systems, ABAC ensures that only authorized doctors and staff can access sensitive patient records based on role and emergency conditions.
- In financial systems, ABAC helps prevent fraud by restricting transactions based on user location, time, and risk level.
- In corporate environments, ABAC controls access to internal systems based on department, project involvement, and employee role.
- In cloud computing, ABAC is used to secure resources like virtual machines and storage services through attribute-based policies.

IDENTITY, CREDENTIALS, AND ACCESS MANAGEMENT (ICAM)

- ICAM is a system that helps organizations **control who can access their data and systems**.
- ICAM is a set of security tools, policies, and systems that helps organizations manage, monitor, and secure access to their IT infrastructure.
- It combines **digital identities, credentials, and access control** into one comprehensive security approach.
- ICAM reduces the risk of cyberattacks by preventing unauthorized access to networks, systems, and data.
- It works in three simple steps:
 1. **Identity** → Who are you? (checking your identity)
 2. **Credentials** → How do you prove it? (password, ID card, login)
 3. **Access Management** → What are you allowed to do? (view, edit, or block access)
- In short: ICAM makes sure **only the right people can access the right things at the right time**.

COMPONENTS OF ICAM

1. Identity Management

- Identity management identifies a subject and verifies that they are who they claim to be before granting system access.
- It manages both physical and digital identities of users.

2. Credential Management

- Credential management binds an identity to authentication methods used for login and verification.
- It includes credentials such as usernames, passwords, and identity cards.
- It ensures that only authenticated users can access systems.

3. Access Management

- Access management defines what users are allowed to see and do within a system.
- It assigns permissions based on roles, groups, and security levels.
- It restricts access to resources unless the user is authenticated and authorized.

BENEFITS OF ICAM

- It improves cybersecurity by allowing access only to authorized users.
- It simplifies user management within organizations.
- It secures sensitive organizational information.
- It enables tracking and monitoring of access to critical data.
- It helps prevent identity fraud and misuse of credentials.

RISKS OF ICAM

- Compromise of sensitive organizational information.
- Spread of misinformation due to unauthorized access.
- Disruption of systems and processes.
- Damage to data integrity and system availability.
- Loss of organizational reputation and credibility.
- Interference with emergency response processes.
- Potential risks to national security in critical systems.

TRUST FRAMEWORKS

- Trust frameworks are shared rules that help different organizations trust each other.
- They reduce the need for separate contracts between organizations.
- They help systems grow easily as more users and organizations join (scalable trust).
- They ensure identity information can be safely shared between systems and users.

COMPONENTS OF TRUST FRAMEWORKS

1. Policies

- Basic rules that all organizations must follow.
- Help decide who can be trusted.

2. Infrastructure

- The technical system that stores and checks identity information.
- Provides a trusted source for verification.

3. Certification

- Process of checking if organizations follow the rules.
- Organizations are approved or rejected based on audits.

COMPONENTS OF TRUST FRAMEWORKS

4. Interoperability

- Allows different systems to work together easily.
- Makes sharing identity information simple and secure.

5. Technical Standards

- Common technical rules for systems to follow.
- Help different systems communicate properly.

6. Legality

- Ensures everything follows laws and regulations.
- Makes identity sharing legally valid and secure.